Flow & Error Control Protocols

Exam Revision Summary — Data Link Layer (Lecture 15 & 16)

1. Protocols — Overview

- Data link layer combines **framing, flow control, and error control** to deliver data reliably between nodes.
- Protocols are implemented in software; described using **pseudocode/algorithms** rather than a specific language.
- Behaviour is modeled using a **Finite State Machine (FSM)**: a machine with finite states, where each event triggers actions and a transition to a next state. One state is the **initial state**.

Taxonomy of Protocols

Channel Type	Protocols
Noiseless Channel	Simplest Protocol, Stop-and-Wait Protocol
Noisy Channel	Stop-and-Wait ARQ, Go-Back-N ARQ, Selective Repeat ARQ

2. Noiseless Channel Protocols

(a) Simplest Protocol

- No flow control, no error control — assumes an **ideal channel** (no loss, duplication, or corruption).
- Receiver can never be overwhelmed, so it handles every frame immediately.
- Sender: gets packet from network layer → makes frame → sends it (repeat forever).
- Receiver: receives frame → extracts packet → delivers to network layer (repeat forever).
- FSM: both sender and receiver have only **one state — 'Ready'**.

(b) Stop-and-Wait Protocol

- Adds **flow control** (still no error control) — sender sends one frame and waits for an ACK before sending the next.
- Sender FSM states: **Ready** (waiting for data) and **Blocking** (waiting for ACK after sending a frame).
- Receiver FSM: single 'Ready' state — on frame arrival, deliver packet and send ACK.
- Purpose: prevent receiver's buffer from being overwhelmed (pure flow control, channel assumed error-free).

3. Noisy Channels — Need for Error Control

- Real channels are noisy — frames can be **lost, corrupted, or duplicated**.
- Three protocols add error control on top of flow control:
- Stop-and-Wait ARQ (Automatic Repeat Request)
- Go-Back-N ARQ
- Selective Repeat ARQ

4. Stop-and-Wait ARQ

- **CRC** added to each frame to detect corruption; corrupted frames are **silently discarded** by receiver.
- Sender starts a **timer** after sending each frame; keeps a **copy** of the frame until ACK arrives.
- If timer **expires** (time-out) → resend the saved frame (assume lost/corrupted).

- If a **corrupted ACK** arrives → discarded (ignored).
- If an **error-free ACK** arrives → stop timer, discard saved copy, send next frame.
- **Sequence numbers**: 0,1,0,1,... (modulo-2). **ACK numbers**: start at 1 — ACK number = sequence number of the *next expected frame*.
- Sender variable **S** = number of most recently sent frame. Receiver variable **R** = number of next expected frame.

Note: Numbered frames & ACKs prevent duplicate frames being accepted — critical when an ACK is delayed and the next frame is lost.

Cases of Operation

- **Normal operation** — sender waits for correct ACK before sending next frame.
- **Lost/damaged frame** — no ACK sent by receiver; sender times out and resends.
- **Lost ACK frame** — receiver discards duplicate frame (frame numbering prevents re-acceptance); sender resends after time-out.
- **Delayed ACK** — a late ACK can cause sender to (wrongly) resend; frame numbering lets receiver detect and discard the duplicate.

Piggybacking: combining a data frame with an ACK in one frame (for bidirectional transmission) — saves bandwidth.

Solved Example — Bandwidth-Delay Product

Bandwidth = 1 Mbps, RTT = 20 ms, Frame size = 1000 bits.
Bandwidth-delay product = $(1\times10^{6}) \times (20\times10^{-3})$ = **20,000 bits**
Utilization = 1000 / 20,000 = **5%** (very inefficient for high-bandwidth or long-delay links)
If protocol allows sending 15 frames before waiting for ACKs: Utilization = 15,000/20,000 = **75%**

Key limitation: Stop-and-Wait has **no pipelining** — must wait for each frame's ACK before sending next → wastes link capacity. Solution: **Sliding Window protocols** (Go-Back-N, Selective Repeat) allow multiple outstanding (sent-but-unacknowledged) frames.

5. Go-Back-N (GBN) ARQ

- Uses a **sliding window** to allow multiple frames "in flight" — improves efficiency via pipelining.
- **Sender window size** must be ≤ $2^m - 1$ (m = bits in sequence number field). If window size = 2^m, duplicate frames can be erroneously accepted.
- **Receiver window size = 1** — receiver only accepts frames in the exact expected order.

Control Variables

Variable	Meaning
S_f	Sequence number of the first (oldest) outstanding/unacknowledged frame
S_n	Sequence number to assign to the next frame to be sent
S_{size}	Send window size = 2^m – 1
R_n	Sequence number of the next frame expected by receiver

Behaviour

- Frames are numbered modulo 2^m.
- Receiver accepts frames **only in order**; an **out-of-order frame is discarded** (no action / no ACK), even if error-free.
- ACKs are **cumulative**: ACK n means "all frames up to n–1 received correctly, expecting frame n next" — one ACK can acknowledge several frames at once.
- On **time-out**, sender resends **ALL outstanding (unacknowledged) frames** — hence the name "Go-Back-N".
- Only **one timer** is used (for the oldest outstanding frame).

Example: Frames 0,1,2,3 sent. Frame 1 is lost. Frames 2 and 3 arrive but are **discarded** (out of order). Sender's timer expires (no ACK received) → sender resends frames 1, 2, and 3 (all outstanding frames).

6. Selective Repeat (SR) ARQ

- More efficient than GBN on noisy links — only the **specific damaged/lost frame** is resent, not all outstanding frames.
- Both **sender and receiver windows** are equal in size; size must be **at most one-half of 2^m** (i.e., ≤ 2^{m-1}), else duplicates can be wrongly accepted.
- Receiver **buffers out-of-order but correct frames** instead of discarding them.
- Each outstanding frame has its **own timer** (multiple timers, one per frame).
- Receiver sends **NAK** for a missing frame (only once per window position, tracked via a nakSent flag, to avoid flooding the network).

Delivery Rule

- Frames are delivered to the network layer only when: (1) a set of **consecutive frames** has arrived, AND (2) that set starts from the **beginning of the window**.
- ACK is sent once data is **delivered** — if several frames are delivered together, only **one ACK** is sent for all of them.

Example: Frames 0,1,2,3 sent; frame 1 is lost. Receiver gets 0 (delivered+ACKed), then 2 and 3 (buffered, NOT delivered — waiting for 1; NAK1 sent once). When frame 1 finally arrives (resent), frames 1, 2, 3 are all delivered together and a single ACK is sent.

7. Quick Comparison Table

Feature	Stop-and-Wait ARQ	Go-Back-N ARQ	Selective Repeat ARQ
Flow control	Yes	Yes (sliding window)	Yes (sliding window)
Error control	Yes	Yes	Yes
Pipelining	No	Yes	Yes
Window size	1	≤ 2m – 1	≤ 2(m-1)
Receiver window	1	1	same as sender
Out-of-order frames	N/A	Discarded	Buffered
Retransmission on error	Resend that frame	Resend ALL outstanding frames	Resend only the bad frame
Timers	1	1	1 per frame
Efficiency (noisy link)	Lowest	Medium	Highest

8. Must-Remember Exam Notes

- Simplest Protocol → **no** flow/error control. Stop-and-Wait Protocol → flow control only (noiseless channel).
- Stop-and-Wait **ARQ** → flow + error control (noisy channel), 1 outstanding frame at a time.
- Sequence numbers in Stop-and-Wait ARQ use **modulo-2** arithmetic (just 0 and 1).
- Go-Back-N: send window ≤ 2^m – 1; receive window = 1; discards out-of-order frames; resends all outstanding on time-out.
- Selective Repeat: window size ≤ $2^{m–1}$ for BOTH sender & receiver; buffers out-of-order frames; resends only the missing frame.
- Piggybacking = combining ACK with outgoing data frame to save bandwidth (used in bidirectional communication).
- Bandwidth-delay product = Bandwidth × Round Trip Time — represents the number of bits that can fill the link ("in the pipe") at once.

End of Summary — Good luck with your exam!